

W15-P1: Create tables category2_43, shop2_43, user2_43, cart2_43, and put 5 products into a cart for the user of your id

=> pgAdmin4, show SQL command to get the needed info

The screenshot shows the pgAdmin4 interface with the following components:

- Object Explorer:** Shows the database structure for 'next_demo_43'. The 'Tables (4)' folder is expanded, showing 'cart2_43', 'category2_43', 'shop2_43', and 'user2_43'.
- Query Editor:** Contains two SQL queries. The first is an INSERT statement for 'cart2_43'. The second is a SELECT query filtering for a specific user and product.
- Data Output:** Displays the results of the SELECT query in a table with 5 rows.

SQL Query 1 (INSERT):

```
INSERT INTO cart2_43 (cid, user_id, product_id, quantity, total, added_at)
VALUES
(11, 213410243, 1, 2, 0, '2023-04-05 10:30:00'),
(12, 213410243, 10, 1, 0, '2023-04-05 11:45:00'),
(13, 213410243, 15, 3, 0, '2023-04-05 12:15:00'),
(14, 213410243, 23, 1, 0, '2023-04-05 13:00:00'),
(15, 213410243, 30, 2, 0, '2023-04-05 14:20:00');
```

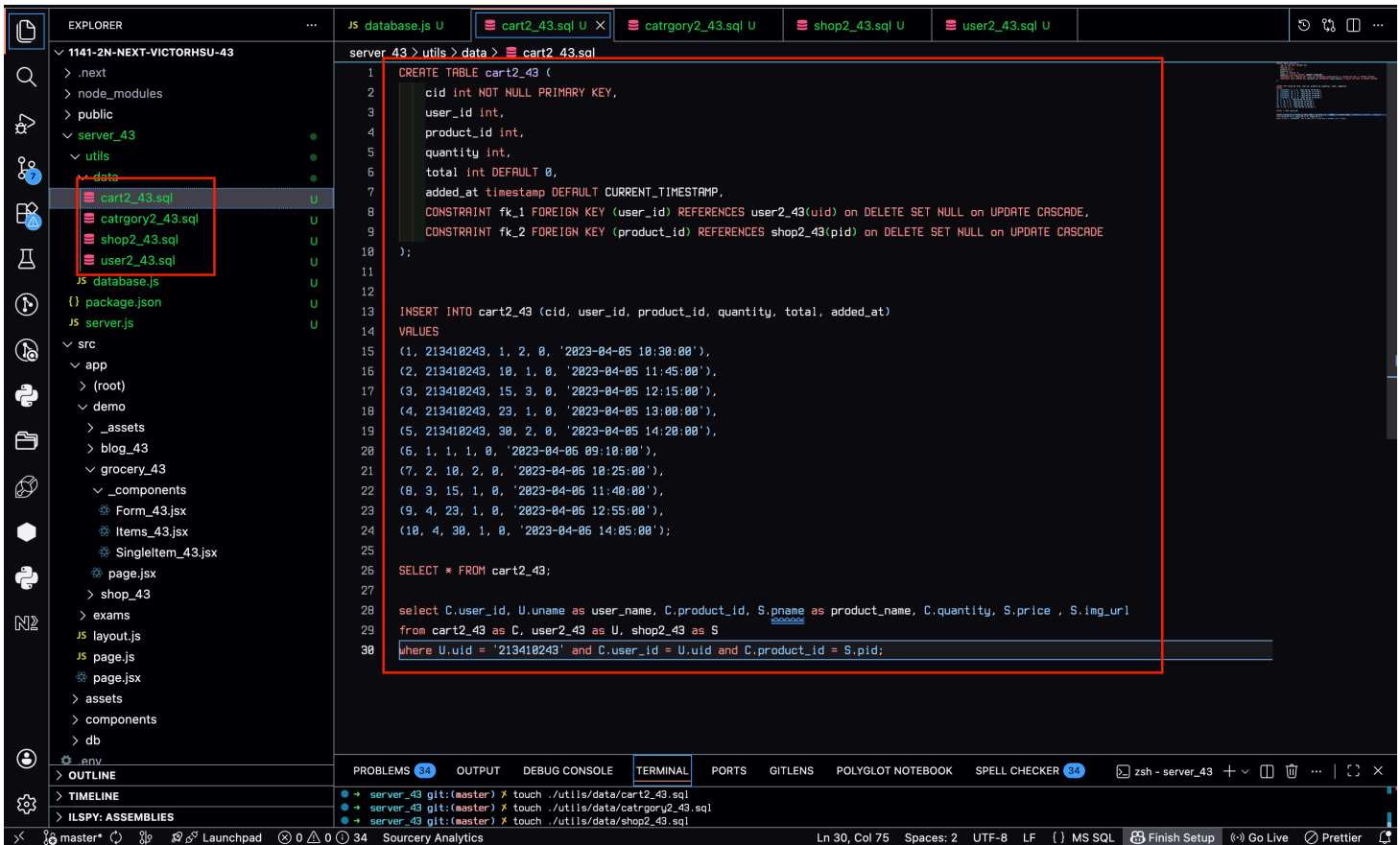
SQL Query 2 (SELECT):

```
select C.user_id, U.name as user_name, C.product_id, S.pname as product_name, C.quant
from cart2_43 as C, user2_43 as U, shop2_43 as S
where U.uid = '213410243' and C.user_id = U.uid and C.product_id = S.pid;
```

Data Output Table:

	user_id integer	user_name character varying (255)	product_id integer	product_name character varying (255)	quantity integer	price real	img_url text
1	213410243	許勝翔	1	Brown Brim	2	25	/images/midterm/hats/brown-brim.png
2	213410243	許勝翔	10	Black Jean Shearling	1	125	/images/midterm/jackets/black-shearling.png
3	213410243	許勝翔	15	Adidas NMD	3	220	/images/midterm/sneakers/adidas-nmd.png
4	213410243	許勝翔	23	Blue Tanktop	1	25	/images/midterm/womens/blue-tank.png
5	213410243	許勝翔	30	Camo Down Vest	2	325	/images/midterm/mens/camo-vest.png

=> sql code



dd20122 victor_xu

Sun Dec 28 15:13:59 2025 +0800 W15-P1: Create tables category2.

W15-P2: Implement route /api/shop_43/cart/:uid to get the info as the SQL in W15-P1

The screenshot shows a VS Code editor with a project named '1141-2N-NEXT-VICTORHSU-43'. The left sidebar shows the Explorer view with a file tree. The main editor area is split into two panes. The left pane shows a REST client with a GET request to 'localhost:5001/api/s...' and a response body containing a JSON array of product information. The right pane shows a SQL query in a file named 'cart2_43.sql'. The query is a SELECT statement that joins the 'cart2_43' table with the 'shop2_43' table to retrieve product details for a specific user ID.

```
SELECT C.user_id, U.name as user_name, C.product_id, S.pname as product_name, C.quantity, S.price,
FROM cart2_43 as C, user2_43 as U, shop2_43 as S
WHERE U.uid = $1 and C.user_id = U.uid and C.product_id = S.pid;
```

61e1898 victor_xu

Sun Dec 28 15:27:34 2025 +0800 W15-P2: Implement route /api/sh

W15-P3: Use localStorage

=> Implement setLocalStorage

The screenshot shows the implementation of the `setLocalStorage` function in the `Items_43.jsx` file. The function is defined as follows:

```
const setLocalStorage = (items) => {  
  localStorage.setItem("list", JSON.stringify(items));  
};
```

The `addItem` function is also shown, which calls `setLocalStorage` when a new item is added:

```
const addItem = (itemName) => {  
  const newItem = {  
    name: itemName,  
    completed: false,  
    id: nanoid(),  
  };  
  const newItems = [...items, newItem];  
  setItems(newItems);  
  setLocalStorage(newItems);  
  toast.success('Item added successfully!');  
};
```

The browser view shows the "Grocery Bud" application with a form to add items and a list of items. The console shows the state of the application, including the items stored in `localStorage`:

Key	Value
list	[{"name": "Egg", "completed": false, "id": "W058CD70fjRjg2p5EKIL"}]

=> Implement getLocalStorage

The screenshot shows the implementation of the `getLocalStorage` function in the `Items_43.jsx` file. The function is defined as follows:

```
const getLocalStorage = () => {  
  if (typeof window !== "undefined") {  
    let list = localStorage.getItem("list");  
    if (list) {  
      list = JSON.parse(list);  
    } else {  
      list = [];  
    }  
    return list;  
  }  
  return []; // Return default for server-side  
};
```

The `useEffect` hook is used to initialize the state from `localStorage` when the component mounts:

```
useEffect(() => {  
  const storedList = getLocalStorage();  
  if (storedList.length > 0) {  
    setItems(storedList);  
  }  
}, []);
```

The browser view shows the "Grocery Bud" application with a form to add items and a list of items. The console shows the state of the application, including the items stored in `localStorage`:

Key	Value
list	[{"name": "Egg", "completed": false, "id": "W058CD70fjRjg2p5EKIL"}]

2b84d60 victor_xu

Sun Dec 28 15:41:45 2025 +0800 W15-P3: Use localStorage

W15-logs: git logs of W15

vic0627 / 1141-2n-next-victorhsu-43

Q Type to search

+

<> Code Issues Pull requests Actions Projects Wiki Security Insights Settings

⚠ Don't get locked out of your account. [Download your recovery codes](#) or [add a passkey](#) so you don't lose access when you get a new device.

Commits

🔍 master

👤 All users

📅 All time

➤ Commits on Dec 28, 2025

W15-P3: Use localStorage

👤 vic0627 committed 4 minutes ago · ✓ 1 / 1

2b84d60

📄 <>

W15-P2: Implement route /api/shop_43/cart/:uid to get the info as the SQL in W15-P1

👤 vic0627 committed 18 minutes ago

61e1898

📄 <>

W15-P1: Create tables category2_43, shop2_43, user2_43, cart2_43, and put 5 products into a cart for the user of your id

👤 vic0627 committed 32 minutes ago

dd20122

📄 <>

➤ Commits on Dec 21, 2025